



SwiftKey gets updated

The new update on the SwiftKey 7 brings in support for customisable stickers and a new toolbar with shortcuts and more. <http://dgit.in/Swftky7>



Google Assistant in Hindi

Google Assistant now gets support for voice commands in Hindi. <http://dgit.in/gassisthindi>

ed using jQuery. So for that we need to install jQuery in our client part of the application.

```
npm install jquery --save
```

Now after adding jQuery we need to import the same into the js file.

```
const $ = require('jquery');
```

After adding jQuery we have to create a function that will take validate and make a call to the backend to save the user in our database. Make sure you haven't put any HTML5 validation in the HTML because till now Electron does not support any HTML5 validation. Now here we want to show some OS dialog box for the window. For that we are going to use Electron JS's native dialog from the remote window. Remote is a part of Electron's renderer process (web view). This lets the main process (app) to communicate with the renderer process.

```
const {dialog} = require('electron').remote;
```

Now after getting the user input from the user we are going to validate via our own validation and then do a post call to the backend server to register the user. Over here we are going to post the user's info to the backend in the post options., then after success we are sending a desktop notification to the user and then automatically logging him/her in. (Code: <https://dgit.in/Dev3>)

Over here `dialog.showErrorBox` is showing the error message from the response. Now the `sendDesktopNotification` function looks like this: <https://dgit.in/Dev4>.

Electron uses HTML5's native notification module to show notifications. Now if the user is successfully registered then he/she is automatically logged in to the system. For that we are calling the `login` function with the user provided email and password. (Code: <https://dgit.in/Dev5>)

Now over here we are again calling the backend system and getting a token in case of successful processing of the user's credentials. Now we need the `auth` token for any further communications with the server. But I will suggest you to have a look at Electron's sessions and cookies to store the data into the cookie, for a better code practice.

Now let us jump back to the back-end to see what's going on over there.

VISITING THE BACK-END

The back-end is mostly an express server with various routes configured within the system. As we have discussed earlier, the main routes are Registration, Login and User. The remaining part is to handle the websocket connection.

Now the registration route is pretty straight forward. Here we are getting

values of fields from the request body then creating and saving new user or denying the request depending on the passed parameters.

The login route is also doing something similar. First, it is finding users on the basis of email passed in the request body. Then it is matching the user's password and then generating a token based on the user's id. That token is a `jwt` token that is passed to the client on success and client is meant to send the token in the header for further authentication at that point (Code: <https://dgit.in/Dev6>)

Now after login the `browserwindow` creates a websocket connection with the back-end. That can be found in the `login.js` in function `connectWebSocket`. In the connect `WebSocket` function the user's id is passed to the back-end so the server can remember the websocket object with an id. (Code: <https://dgit.in/Dev7>)

Now after log in the user will see a box to search for a friend and if that person is online then the system will connect the two by opening up the chat box. Now the first screen after login looks like this. This is where you can enter the email ID of another registered user to try and initiate a chat with them, but to make that happen needs a little more to be done in the back-end of the project.

*POINTERS



* No-fuss RSS vulture app using Electron

Creating a custom RSS headline scanning app for journalists and news junkies using the Electron framework <https://dgit.in/ElectronRSS>



*How to create a hybrid Electron app

Create an Electron app that loads your web app and adds native functionality to the same. <https://dgit.in/HybridElectron>



*Electron Apps

A directory of all kinds of apps created with Electron.js organised by categories like productivity, games, developer tools and many more. <https://dgit.in/ElectronApps>